

Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики
Кафедра інтелектуальних програмних систем

ЗВІТ
з виробничої практики з відривом від навчання
з освітньо-професійної програми «Програмна інженерія»
за спеціальністю 121 Інженерія програмного забезпечення

Виконала
Студентка 3-го курсу
КЛЕВЧУК Марія Вячеславівна

Керівник практики від кафедри:
асистент
СУПРУН Олег Олексійович

Керівник практики від бази
практики:
Голова ГО «Центр цифрових
досліджень»
МАКАРЕНКО Максим Олегович

Засвідчую, що в цій роботі
немає запозичень з праць
інших авторів без відповідних
посилань

Студентка _____

ЗМІСТ

ВСТУП	3
РОЗДІЛ 1 МЕТА ТА ЗАВДАННЯ ПРАКТИКИ НА БАЗІ ПРАКТИКИ.....	5
РОЗДІЛ 2 НАПРЯМИ ТА ХАРАКТЕРИСТИКА ДІЯЛЬНОСТІ БАЗИ ПРАКТИКИ	7
РОЗДІЛ 3 ДОСЛІДЖЕННЯ, АНАЛІЗ І ОБҐРУНТУВАННЯ ТЕХНІЧНОГО СТЕКУ	8
РОЗДІЛ 4 ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА РЕАЛІЗАЦІЯ ЛОГІКИ ...	10
ВИСНОВКИ	12
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ	13

ВСТУП

Сучасний етап розвитку глобальної інформаційної індустрії характеризується стрімкою еволюцією вебтехнологій та переходом від традиційних статичних ресурсів до складних високофункціональних односторінкових застосунків (Single Page Applications, SPA). В умовах постійного зростання обсягів даних та підвищення вимог користувачів до швидкодії інтерфейсів, питання проектування та розробки якісної фронтенд-частини програмних продуктів стає критично важливим для успішного функціонування будь-якого підприємства. Ефективність внутрішніх інформаційних систем безпосередньо корелює з продуктивністю персоналу, швидкістю обробки бізнес-операцій та мінімізацією помилок, зумовлених людським фактором.

Актуальність даної виробничої практики зумовлена необхідністю глибокого вивчення та практичного впровадження сучасних інструментів розробки, таких як бібліотека React.js та фреймворк Tailwind CSS, для модернізації застарілих інтерфейсів. Більшість існуючих корпоративних систем, розроблених на базі архітектур попередніх поколінь, стикаються з проблемою низької адаптивності та надлишкового навантаження на сервер через постійне повне перезавантаження сторінок. Перехід до компонентно-орієнтованої архітектури дозволяє не лише підвищити швидкість відгуку системи за рахунок використання механізму Virtual DOM, а й забезпечити високу масштабованість та легкість підтримки програмного коду в довгостроковій перспективі.

Виробнича практика є фундаментальним етапом підготовки бакалавра за спеціальністю. Вона надає можливість інтегрувати теоретичні знання, здобуті під час академічного навчання, у реальний виробничий цикл розробки програмного забезпечення. Проходження практики на базі реального підприємства дозволяє студенту ознайомитися з професійними стандартами написання коду, методологіями управління проектами (Agile, Scrum) та інструментарієм командної роботи. Окрему увагу в рамках практики приділено дослідженню принципів побудови адаптивних інтерфейсів (Responsive Web Design), що є стандартом де-факто.

Об'єктом проходження практики є технологічний процес проектування та реалізації клієнтської частини (front-end) інформаційної системи обліку та моніторингу.

Предметом дослідження є методи, засоби та програмні інструменти побудови інтерфейсів користувача із використанням декларативного підходу, сучасних інструментів збірки проектів та бібліотек управління станом додатка.

Методологічну основу звіту складають принципи системного аналізу, об'єктно-орієнтованого проектування та компонентного моделювання програмних систем. Під час виконання завдань практики особлива увага

приділялася дотриманню інженерних принципів SOLID та DRY (Don't Repeat Yourself), що є базовими для створення надійного та чистого програмного коду. Даний звіт містить детальний опис усіх етапів розробки: від передпроектного аналізу предметної області до фінального тестування та оцінки продуктивності реалізованого модуля.

1. МЕТА ТА ЗАВДАННЯ ПРАКТИКИ НА БАЗІ ПРАКТИКИ

Головною метою виробничої практики є закріплення та поглиблення теоретичних знань, отриманих під час вивчення базових та спеціальних дисциплін, а також набуття практичних навичок і професійних компетенцій у сфері аналізу, розробки, тестування та супроводу сучасних програмних систем. Особливий акцент ставиться на вивченні технологічного циклу створення фронтенд-частини вебзастосунків із використанням передового інструментарію розробки.

Для досягнення поставленої мети в ході проходження практики необхідно було розв'язати наступний комплекс пріоритетних завдань.

Ознайомчий етап та аналіз середовища:

- Вивчення організаційної структури бази практики, функціональних обов'язків персоналу IT-відділу та регламентів внутрішньої комунікації.
- Проходження інструктажу з техніки безпеки при роботі з комп'ютерною технікою та ознайомлення з нормативно-правовою базою захисту інформації на підприємстві.

Передпроектне дослідження та аудит:

- Проведення детального системного аналізу існуючого програмного забезпечення підприємства.
- Ідентифікація технічних та архітектурних проблем у поточному користувацькому інтерфейсі (UI), аналіз показників доступності та швидкодії.

Науково-технічне обґрунтування рішень:

- Дослідження сучасних тенденцій розвитку Front-end розробки та проведення порівняльного аналізу доступних бібліотек і фреймворків.
- Вибір та наукове обґрунтування оптимального технологічного стеку (React.js, Vite, Tailwind CSS) відповідно до специфіки поставленої задачі.

Проектно-конструкторський етап:

- Розробка архітектурної схеми фронтенд-модуля на основі принципів низької зв'язності та високої зв'язності
- Проектування компонентної структури застосунку та визначення механізмів управління станами (State Management).

Етап практичної реалізації:

- Розробка адаптивного, кросбраузерного інтерфейсу користувача з використанням обраних інструментів стилізації.

- Імплементація логіки взаємодії з серверною частиною через RESTful API, налаштування механізмів обробки помилок та валідації вхідних даних.

Верифікація та документація:

- Проведення комплексного тестування розробленого модуля на відповідність функціональним та нефункціональним вимогам.
- Систематизація отриманих результатів та оформлення фінального звіту про проходження практики згідно з вимогами державного стандарту.

2. НАПРЯМИ ТА ХАРАКТЕРИСТИКА ДІЯЛЬНОСТІ БАЗИ ПРАКТИКИ

Виробнича практика проходила на базі ГО «Центр цифрових досліджень», яка працює у сфері інформаційних технологій та спеціалізується на розробці, впровадженні та технічній підтримці прикладного програмного забезпечення для автоматизації бізнес-процесів. Діяльність бази практики охоплює повний цикл створення програмних продуктів, зокрема: проектування та розробка вебзастосунків, технічна підтримка та супровід, ІТ-консалтинг та аудит.

Компанія побудована за принципом функціональної структури, де ключову роль відіграє відділ розробки програмного забезпечення. Внутрішня взаємодія між фахівцями базується на використанні гнучких методологій розробки.

Особливості виробничого середовища:

- Система контролю версій: Увесь програмний код зберігається та адмініструється за допомогою розподіленої системи Git, що забезпечує прозорість змін та можливість паралельної роботи над модулями.
- Управління проектами: Для відстеження життєвого циклу завдань використовуються сучасні таск-менеджери, де фіксуються дедлайни, пріоритети та технічні вимоги.
- Середовище розробки: Використання контейнеризації (Docker) та сучасних IDE дозволяє підтримувати ідентичність середовища розробки на всіх етапах — від локальної машини програміста до робочого сервера.

Ознайомлення з діяльністю підприємства дозволило зрозуміти вимоги ринку до сучасного інженера-програміста та сформуванати перелік необхідних компетенцій для успішної реалізації завдань практики.

3. ДОСЛІДЖЕННЯ, АНАЛІЗ І ОБҐРУНТУВАННЯ ТЕХНІЧНОГО СТЕКУ

Процес розробки сучасного фронтенд-модуля вимагає ретельного підбору технологічного інструментарію, який забезпечить не лише швидку реалізацію поточних завдань, а й масштабованість та легкість підтримки коду в майбутньому

3.1. Аналіз та вибір базового JavaScript-фреймворку

Основним критерієм вибору фронтенд-бібліотеки була здатність ефективно працювати з динамічними даними та підтримувати компонентно-орієнтовану архітектуру. Було розглянуто три найбільш розповсюджені рішення: React, Vue.js та Angular.

Angular є повноцінним фреймворком з жорсткою структурою, що забезпечує високу типізацію, проте він має високий поріг входу та є дещо надлишковим для модернізації окремого інтерфейсного модуля.

Vue.js пропонує прогресивний підхід та легкість інтеграції, але його екосистема в корпоративному сегменті є дещо меншою порівняно з конкурентами.

React.js було обрано як найбільш оптимальне рішення для даного проекту. Основною перевагою React є декларативний підхід до опису UI та використання механізму Virtual DOM, що дозволяє оновлювати лише необхідні частини інтерфейсу без повного перемальовування сторінки. Це критично важливо для продуктивності систем обліку з великою кількістю таблиць та форм. Крім того, архітектура на основі функціональних компонентів та хуків (Hooks) сприяє створенню чистого та перевикористовуваного коду, що повністю відповідає інженерному принципу DRY (Don't Repeat Yourself).

3.2. Обґрунтування вибору інструменту збірки проекту

Для забезпечення швидкого циклу розробки (Development Cycle) та ефективної оптимізації фінального продукту було проведено аналіз інструментів збірки (Bundlers). Традиційний Webpack, попри свою універсальність, демонструє низьку швидкість "гарячого перезавантаження" (Hot Module Replacement) на великих проектах.

У даній роботі було обрано Vite. На відміну від попередників, Vite використовує нативні ES-модулі браузера під час розробки, що забезпечує миттєвий запуск сервера та швидке відображення змін. Для фінальної збірки Vite використовує Rollup, який генерує максимально оптимізовані статичні файли, зменшуючи час завантаження сторінки для кінцевого користувача. Це рішення дозволило скоротити час підготовки середовища розробки та підвищити загальну ефективність роботи.

3.3. Вибір методології та інструментів стилізації

При проектуванні дизайну інтерфейсу ключовими вимогами були адаптивність, консистентність та швидкість ітерацій. Було розглянуто

стандартний CSS (з використанням BEM-методології) та сучасні Utility-first рішення.

Вибір було зроблено на користь Tailwind CSS. Цей фреймворк кардинально відрізняється від традиційних підходів тим, що надає набір низькорівневих утилітарних класів, які комбінуються безпосередньо в HTML/JSX-розмітці. Такий підхід має ряд інженерних переваг:

- Мінімізація об'єму CSS-коду: Tailwind генерує стилі лише для тих класів, що фактично використовуються, що позитивно впливає на показники Google Lighthouse.
- Легкість підтримки адаптивності: Завдяки вбудованій системі префіксів (sm:, md:, lg:) реалізація інтерфейсів для мобільних пристроїв та планшетів стає частиною процесу верстки, а не окремим етапом.
- Дизайн-система: Використання зумовленої палітри кольорів та сітки відступів гарантує візуальну цілісність усіх модулів системи без необхідності постійної синхронізації стилів.

3.4. Додаткові інструменти та бібліотеки

Для забезпечення надійної взаємодії з серверною частиною (Backend API) було обрано бібліотеку Axios. Вона була віддана перевага над стандартним API fetch завдяки вбудованому механізму перехоплювачів (interceptors), що дозволяє централізовано додавати токени авторизації до кожного запиту та обробляти помилки в одному місці.

Для управління маршрутизацією було використано React Router, що дозволило створити логічну структуру переходів між сторінками без перезавантаження браузера, забезпечуючи повноцінний SPA-досвід для користувача.

Таким чином, сформований стек технологій (React + Vite + Tailwind CSS + Axios) є збалансованим інженерним рішенням, яке забезпечує високу швидкість розробки, відмінну продуктивність клієнтської частини та простоту супроводження програмного продукту.

4. ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА РЕАЛІЗАЦІЯ ЛОГІКИ ІНТЕРФЕЙСУ

Процес розробки оновленого фронтенд-модуля базувався на принципах модульності, масштабованості та низької зв'язності компонентів. Головною метою проектування було створення архітектури, яка дозволить легко додавати новий функціонал без необхідності рефакторингу існуючих частин системи.

4.1. Структурна організація проєкту

Для забезпечення чистоти коду та зручності навігації було впроваджено чітку ієрархію папок у каталозі `src`. Кожен тип файлів має своє строго визначене місце, що відповідає стандартам промислової розробки на React:

`/components`: містить перевикористовувані UI-компоненти (Buttons, Inputs, Modals, Tables). Кожен компонент ізольований і не залежить від глобального стану напрямку.

`/pages`: високорівневі компоненти, що представляють цілі сторінки додатку (Dashboard, Login, Analytics).

`/hooks`: кастомні React-хуки, в які винесена повторювана логіка.

`/services`: конфігурація Axios та функції для взаємодії з API-ендпоінтами.

`/context`: реалізація глобального стану додатка через React Context API.

Така структура дозволяє мінімізувати дублювання коду та забезпечує дотримання принципу Separation of Concerns (розділення відповідальності).

4.2. Розробка компонентної моделі

В основі інтерфейсу лежить методологія розділення компонентів на «презентаційні» (dumb) та «контейнерні» (smart).

Презентаційні компоненти відповідають виключно за візуальне відображення. Вони отримують дані та функції через props і не знають про існування серверної частини чи глобального стану. Це робить їх легкотестованими та універсальними.

Контейнерні компоненти (сторінки) акумулюють у собі бізнес-логіку: роблять запити до API, обробляють стан завантаження та передають чисті дані вниз по дереву компонентів.

Для стилізації використано Tailwind CSS, що дозволило впровадити адаптивність на рівні кожного окремого компонента. Використання сітки Flexbox та CSS Grid забезпечило коректне розташування елементів на екранах із різною роздільною здатністю.

4.3. Реалізація логіки взаємодії з Backend API

Одним із найскладніших завдань була організація надійного обміну даними з

сервером. Для цього було використано бібліотеку Axios із налаштуванням централізованого інстансу.

Ключові аспекти реалізації:

Інтерцептори (Interceptors): Було налаштовано перехоплювач запитів, який автоматично додає JWT-токен до заголовків (Headers) кожного запиту, якщо користувач авторизований. Також налаштовано перехоплювач відповідей для глобальної обробки помилок.

Обробка станів (Loading & Error states): Для покращення UX було реалізовано динамічне відображення індикаторів завантаження та модальних вікон із повідомленнями про помилки мережі.

Валідація вхідних даних: Реалізовано багаторівневу валідацію форм на клієнтській стороні. Використання регулярних виразів та перевірки типів даних дозволяє відсіяти некоректні запити ще до їх відправки на сервер, що знижує навантаження на систему та пришвидшує взаємодію для користувача.

4.4. Маршрутизація та захист даних

Для управління навігацією застосовано бібліотеку React Router v6.

Проектування маршрутів включало створення ієрархічної структури:

Public Routes: доступні для всіх (сторінка входу, відновлення пароля).

Private Routes: захищені маршрути, доступ до яких мають лише автентифіковані співробітники. Реалізація здійснюється через компонент-обгортку, який перевіряє стан авторизації в глобальному контексті та, у разі відсутності прав, перенаправляє користувача на сторінку Login.

4.5. Оптимізація продуктивності

Для забезпечення високої швидкодії було впроваджено декілька інженерних рішень:

Code Splitting: використання `React.lazy()` та `Suspense` для завантаження коду сторінок лише тоді, коли користувач на них переходить. Це значно зменшило розмір початкового JS-бандлу.

Memoization: використання хуків `useMemo` та `useCallback` для запобігання зайвих рендерингів важких компонентів (наприклад, великих таблиць із даними замовлень).

Image Optimization: налаштування автоматичного стиснення зображень та використання сучасних форматів (WebP) через конфігурацію Vite. Результатом реалізації цієї архітектури став стабільний, швидкий та адаптивний інтерфейс, який легко піддається подальшому масштабуванню та модернізації.

ВИСНОВКИ

У ході виконання завдань практики було реалізовано повний цикл модернізації фронтенд-модуля внутрішньої інформаційної системи, що дозволило отримати неоціненний досвід роботи з реальним програмним продуктом.

Головними результатами проходження практики є:

Аналітична підготовка: Проведено детальний аудит існуючої архітектури системи, в ході якого було виявлено критичні недоліки в продуктивності та юзабіліті застарілого інтерфейсу. Це дозволило сформулювати чітке технічне завдання на розробку оновленого модуля.

Обґрунтування технологій: На основі порівняльного аналізу було обрано та впроваджено сучасний стек технологій — React.js, Vite та Tailwind CSS. Вибір цих інструментів довів свою ефективність, забезпечивши високу швидкість розробки та відмінні показники швидкодії (Performance) фінального продукту.

Архітектурне проектування: Спроектовано та реалізовано модульну структуру проекту, що базується на принципах чистого коду (Clean Code) та компонентно-орієнтованому підході. Впровадження перевикористовуваних UI-компонентів дозволило скоротити час на розробку нових інтерфейсних рішень та забезпечило візуальну цілісність системи.

Програмна реалізація: Успішно імплементовано логіку взаємодії з серверною частиною через REST API за допомогою бібліотеки Axios. Налаштовано захищену маршрутизацію та багаторівневу валідацію даних, що підвищило надійність та безпеку роботи користувачів із системою.

Оптимізація та адаптивність: Завдяки використанню методології Responsive Web Design та інструментів оптимізації (Code Splitting, Lazy Loading), створений інтерфейс коректно функціонує на всіх типах пристроїв, демонструючи стабільно високу швидкість завантаження.

Окрім технічних аспектів, практика сприяла розвитку (soft skills): вмінню працювати з системою контролю версій Git, дотриманню дедлайнів та взаємодії в межах професійного IT-середовища. Робота над проектом підтвердила важливість дотримання інженерних стандартів та принципів SOLID при побудові масштабованих систем.

Підсумовуючи, можна стверджувати, що мета практики досягнута, а всі поставлені завдання виконані в повному обсязі. Отримані результати та практичні навички стануть надійним фундаментом для майбутньої професійної діяльності у сфері розробки програмного забезпечення.

ПЕРЕЛІК ДЖЕРЕЛ ТА ПОСИЛАННЯ

- React Official Documentation. [Електронний ресурс]. <https://react.dev/>
- Tailwind CSS: Utility-First Fundamentals. [Електронний ресурс]. <https://tailwindcss.com/docs/>
- Vite: Next Generation Frontend Tooling. [Електронний ресурс]. <https://vitejs.dev/guide/>
- Axios HTTP Client: Getting Started. [Електронний ресурс]. <https://axios-http.com/docs/intro>
- React Router v6 Documentation. [Електронний ресурс]. <https://reactrouter.com/en/main>
- MDN Web Docs: Web APIs & DOM. [Електронний ресурс]. <https://developer.mozilla.org/en-US/docs/Web/API>
- SOLID Principles in React Applications. [Електронний ресурс]. <https://www.freecodecamp.org/news/solid-principles-for-programmers/>
- JWT (JSON Web Tokens) Authentication Guide. [Електронний ресурс]. <https://auth0.com/learn/json-web-tokens/>
- Web Content Accessibility Guidelines (WCAG) 2.1. [Електронний ресурс]. <https://www.w3.org/WAI/standards-guidelines/wcag/>
- Responsive Web Design Patterns (Google Developers). [Електронний ресурс]. <https://web.dev/responsive-web-design-basics/>